LAPORAN TUGAS 2 IF4020

Kriptografi

Perancangan dan Analisis Brainrot Cipher

Kelompok 11



Disusun oleh

Muhammad Ra'if Alkautsar	13523011
Muhammad Aditya Rahmadeni	13523028
Fityatul Haq	13523116

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESHA 10, BANDUNG 40132

2026/2027

PERNYATAAN TIDAK MELAKUKAN KECURANGAN

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.



Muhammad Ra'if Alkautsar



Muhammad Aditya Rahmadeni



Fityatul Haq

DAFTAR ISI

PERNYATAAN TIDAK MELAKUKAN KECURANGAN . . . . . **i**

BAB I PENDAHULUAN **1**

1.1 Latar Belakang 1

BAB II DASAR TEORI **2**

2.1 Block Cipher dan SPN 2

2.2 S-box, Permutasi, dan Operasi Tambahan 2

2.3 Pembentukan Kunci Putaran 2

2.4 Mode Operasi. 3

2.5 Padding dan Verifikasi Data 3

2.6 Ukuran Statistik 3

BAB III PERANCANGAN BLOCK CIPHER **5**

3.1 Parameter dan Representasi State 5

3.2 Komponen Transformasi 5

3.2.1 SubBytes67 5

3.2.2 ShiftRows67 7

3.2.3 MixColumns67 7

3.2.4 Bit-Plane Transpose (BPT) 7

3.2.5 Feistel Cross-Mixing (FCM) 8

3.2.6 Penjumlahan Modulo Byte 8

3.2.7 AddRoundKey67. 8

3.3 Alur Enkripsi dan Dekripsi 8

3.4 Pembentukan Kunci Putaran 9

BAB IV IMPLEMENTASI DAN SIMULASI **10**

4.1 Frontend, Backend, dan CLI 10

4.2 Demonstrasi Teks dan Berkas Biner 10

4.3 Justifikasi dan Bukti Padding 11

4.4 Pengelolaan IV dan Counter 11

4.5 Bukti Pemulihan Data dan Batas Integritas 12

BAB V ANALISIS KEAMANAN **13**

5.1 Konfigurasi dan Reproduksibilitas 13

5.2 Avalanche Effect Satu Blok 13

5.3 Avalanche Effect pada Seluruh Mode 14

5.4 Analisis Entropi 14

5.5 Analisis Histogram 15

5.6 Pengujian Tambahan dan Keterbatasan 18
5.6.1 Kebenaran Implementasi 18
BAB VI KESIMPULAN DAN SARAN PENGEMBANGAN 19
6.1 Kesimpulan 19
6.2 Saran Pengembangan 20
DAFTAR PUSTAKA 22
LAMPIRAN 23
A. Pranala Repositori 23
B. Video Demo dan Dokumentasi API (Opsional) 23
C. Pembagian Tugas Anggota 23
D. Petunjuk Menjalankan 24

DAFTAR GAMBAR

3.1 Alur enkripsi satu blok sesuai implementasi saat ini. 9

5.1 Histogram fixture teks dan cipherteks lima mode pada seed 42; sumbu y memakai skala berbeda dan garis putus-putus menandai distribusi seragam. 16

5.2 Histogram fixture biner dan cipherteks lima mode, seed 42; sumbu y memakai skala berbeda. 17

DAFTAR TABEL

3.1 Parameter implementasi Brainrot Cipher. 5
3.2 Contoh hasil S-box baru dan pemetaan inversnya. 6
3.3 Offset permutasi baris dalam implementasi. 7

4.1 Pemulihan berkas melalui endpoint HTTP dengan PKCS#7. 11
4.2 Pemeriksaan batas blok pada CBC dengan PKCS#7. 11
4.3 Pengujian perubahan data. 12

5.1 Hasil avalanche Brainrot satu blok, 256 percobaan dan seed 42. 13
5.2 Avalanche pesan teks 404 byte, 256 percobaan dan seed 42. 14
5.3 Entropi empiris untuk fixture teks, seed 42; ukuran adalah badan cipherteks. 15
5.4 Entropi empiris untuk fixture biner, seed 42. 15

6.1 Isian pembagian tugas kelompok. 23

BAB I

PENDAHULUAN

1.1. Latar Belakang

Perkembangan pertukaran informasi digital menimbulkan kebutuhan untuk melindungi kerahasiaan data, baik dalam bentuk teks maupun berkas biner. Salah satu pendekatan yang digunakan adalah kriptografi simetris, yaitu proses enkripsi dan dekripsi menggunakan kunci rahasia yang sama. Block cipher merupakan algoritma kriptografi simetris yang memproses data dalam blok berukuran tetap. Untuk menangani pesan yang lebih panjang, block cipher digunakan bersama mode operasi tertentu.

DES dan AES menjadi pembanding dalam perancangan algoritma ini. DES menggunakan blok 64 bit, kunci efektif 56 bit, dan struktur Feistel dengan 16 putaran. DES merupakan standar historis yang telah ditarik oleh NIST. Sementara itu, AES menggunakan blok 128 bit dengan pilihan panjang kunci 128, 192, atau 256 bit, masing-masing menggunakan 10, 12, atau 14 putaran. AES menggabungkan substitusi byte, pergeseran baris, pencampuran kolom, dan penambahan kunci putaran. Rancangan Brainrot Cipher lebih dekat dengan pendekatan AES daripada DES. Referensi pembanding yang digunakan adalah spesifikasi DES dan spesifikasi AES.

Brainrot Cipher dirancang sebagai block cipher eksperimental berbasis Substitution–Permutation Network atau SPN. Algoritma ini menggunakan blok 128 bit, kunci 128 bit, dan 10 putaran. Gagasan utamanya adalah menggabungkan substitusi nonlinear dan transformasi linear yang dapat dibalik, kemudian mengikat setiap putaran dengan kunci yang diturunkan dari kunci utama. Karakteristik khususnya terdapat pada modifikasi Kotak-S, pola pergeseran baris, matriks pencampuran kolom, serta penggunaan konstanta tertentu pada pembentukan dan penambahan kunci putaran.

BAB II

DASAR TEORI

2.1. Block Cipher dan SPN

Block cipher memetakan blok plainteks berukuran tetap menjadi blok cipherteks dengan kunci rahasia. Untuk setiap kunci K, enkripsi E_K memiliki invers D_K sehingga $D_K(E_K(P)) = P$. *Substitution–Permutation Network* (SPN) mengulang lapisan substitusi, permutasi atau pencampuran, dan penggabungan kunci. Dekripsi menerapkan invers setiap transformasi dalam urutan terbalik. Struktur dasar tersebut dijelaskan dalam kajian SPN oleh Heys [6].

Shannon memperkenalkan prinsip *confusion* dan *diffusion* untuk mempersulit analisis statistik cipher [5]. *Confusion* membuat hubungan antara statistik cipherteks dan kunci menjadi kompleks, sedangkan *diffusion* menyebarkan struktur statistik plainteks ke banyak bagian cipherteks. Dalam Brainrot, substitusi nonlinier dan penggabungan kunci mendukung *confusion*, sedangkan pergeseran baris serta pencampuran kolom mendukung *diffusion*. Keduanya merupakan prinsip perancangan, bukan bukti bahwa suatu cipher telah aman.

2.2. S-box, Permutasi, dan Operasi Tambahan

S-box memetakan nilai byte menggunakan tabel. Pemetaan bijektif memiliki tabel invers untuk dekripsi. Permutasi mengubah posisi byte tanpa mengubah nilainya. Brainrot memakai XOR, pergeseran melingkar posisi byte, rotasi bit, pencampuran pada GF(2^8), penjumlahan modulo, dan pencampuran Feistel. Langkah Feistel dapat dibalik tanpa mensyaratkan fungsi internal bijektif. Perkalian medan dalam kode memakai polinomial reduksi $x^8 + x^4 + x^3 + x + 1$, yang juga digunakan AES [2].

Pada medan tersebut, satu byte merepresentasikan polinomial biner berderajat paling tinggi tujuh. Penjumlahan dilakukan dengan XOR, sedangkan perkalian diikuti reduksi polinomial, bukan perkalian bilangan bulat biasa. Matriks pencampuran harus memiliki invers agar kolom state dapat dipulihkan. XOR dengan nilai yang sama membalik operasi sebelumnya, $(x \oplus k) \oplus k = x$, sedangkan pergeseran melingkar dibalik dengan pergeseran berlawanan arah.

2.3. Pembentukan Kunci Putaran

Key schedule menurunkan kunci-kunci putaran secara deterministik dari kunci utama. AES menggunakan rotasi word, substitusi, XOR, dan konstanta putaran dalam proses perluasan kunci [2]. Pendekatan ini menjadi acuan Brainrot, yang menghasilkan 21 subkunci berukuran 16 byte: satu untuk whitening dan dua untuk setiap putaran, yaitu penjumlahan modulo dan XOR. Dekripsi memakai subkunci yang sesuai dalam urutan operasi dan putaran terbalik. Perluasan ini tidak menambah entropi kunci utama; seluruh kunci putaran tetap ditentukan oleh kunci utama 128 bit. Rincian modifikasinya dijelaskan pada Bab III.

2.4. Mode Operasi

ECB, CBC, CFB, OFB, dan CTR adalah lima mode kerahasiaan yang dijelaskan pada NIST SP 800-38A [3]. ECB memproses blok secara independen; CBC mengaitkan blok melalui cipherteks sebelumnya. CFB memakai umpan balik cipherteks, OFB memakai umpan balik keluaran cipher, dan CTR mengenkripsi nilai pencacah. Rincian implementasi dan penanganan blok akhir tersedia pada Bab IV.

ECB tidak menggunakan *initialization vector* (IV), sehingga blok plainteks yang sama menghasilkan blok cipherteks yang sama untuk kunci yang sama. CBC dan CFB memerlukan IV yang tidak dapat diprediksi; OFB memerlukan IV yang unik untuk setiap pesan dengan kunci yang sama. Pada CTR, seluruh blok pencacah harus unik untuk kunci yang sama, termasuk antarberkas atau antarpesan. IV tidak perlu dirahasiakan, tetapi harus memenuhi persyaratan mode yang digunakan [3].

2.5. Padding dan Verifikasi Data

ECB dan CBC memerlukan masukan berupa blok lengkap. Untuk panjang blok b byte dan pesan L byte, PKCS#7 menambahkan $p = b - (L \bmod b)$ byte, masing-masing bernilai p. Pesan yang sudah tepat kelipatan blok tetap mendapat satu blok padding sehingga penghapusannya tidak ambigu [4]. Pada dekripsi, nilai p harus berada pada rentang 1 sampai b dan seluruh p byte terakhir harus bernilai p. Dalam implementasi ini, CFB, OFB, dan CTR memproses sisa byte melalui XOR dengan keystream tanpa padding.

Kesamaan byte hasil dekripsi dengan masukan membuktikan pemulihan data pada pengujian tersebut, bukan autentikasi integritas. Padding yang valid juga tidak membuktikan bahwa cipherteks belum dimodifikasi. Deteksi manipulasi memerlukan mekanisme autentikasi, misalnya *Message Authentication Code* (MAC), yaitu tag yang dihitung menggunakan kunci rahasia dan diverifikasi oleh penerima [4, bagian 9]. Implementasi Brainrot saat ini belum menyertakan MAC atau tag autentikasi.

2.6. Ukuran Statistik

Avalanche effect diamati dengan mengubah tepat satu bit plainteks atau kunci, sementara parameter lainnya dipertahankan. Jika dua cipherteks C dan C' memiliki panjang yang sama, yaitu $N > 0$ bit, persentase perubahan bit dinyatakan sebagai

$$A(C, C') = \frac{\mathrm{HW}(C \oplus C')}{N} \times 100\%, \tag{2.1}$$

dengan HW menyatakan banyaknya bit bernilai satu. Untuk satu blok Brainrot, $N = 128$. Rata-rata mendekati 50% menunjukkan bahwa sekitar separuh bit keluaran berubah pada sampel pengujian, tetapi tidak membuktikan ketahanan terhadap kriptanalisis.

Entropi empiris byte dihitung dari frekuensi relatif p_v setiap nilai byte v:

$$H = -\sum_{v=0}^{255} p_v \log_2 p_v, \qquad 0 \log_2 0 = 0. \tag{2.2}$$

Nilai maksimum distribusi byte adalah 8 bit/byte, tercapai ketika seluruh 256 nilai byte memiliki frekuensi relatif yang sama. Histogram menampilkan jumlah kemunculan masing-masing nilai v. Entropi tinggi dan histogram merata tidak menjamin tidak adanya pola atau korelasi antarbyte.

Pengukuran satu blok harus dibedakan dari pengukuran seluruh pesan. Sebagai contoh, pada OFB dan CTR dengan kunci serta IV tetap, perubahan satu bit plainteks hanya mengubah satu bit cipherteks karena keystream yang digunakan tetap sama. Oleh karena itu, ukuran statistik perlu ditafsirkan bersama ukuran sampel dan sifat mode operasi.

BAB III

PERANCANGAN BLOCK CIPHER

3.1. Parameter dan Representasi State

Tabel 3.1. Parameter implementasi Brainrot Cipher.

Parameter	Nilai
Struktur	Substitution-Permutation Network (SPN)
Ukuran blok	128 bit (16 byte)
Kunci utama	128 bit (16 byte)
Jumlah putaran	10
Kunci hasil ekspansi	21 subkunci, Q_0 sampai Q_{20}
Putaran ganjil / genap	6 / 7 operasi; MixColumns67 selalu digunakan
State	Matriks 4×4 byte, diisi per kolom

Byte masukan P_i dipetakan ke $s_{r,c} = P_{4c+r}$, untuk $0 \leq r, c < 4$. Serialisasi keluaran menggunakan urutan yang sama.

3.2. Komponen Transformasi

3.2.1. SubBytes67

SubBytes67 mengganti setiap byte state menggunakan tabel substitusi S_{67} yang dibentuk dari permutasi matriks 16×16. Fungsi `generateSbox67` pada `src/cipher/brainrot/tables.go` mula-mula mengisi matriks $B^{(0)}$ dengan nilai 0 sampai 255 secara berurutan per baris:

$$B^{(0)}_{r,c} = 16r + c, \qquad 0 \leq r, c < 16. \tag{3.1}$$

Semua indeks dimulai dari nol. Jadi, indeks 6 dan 7 menunjukkan baris atau kolom ketujuh dan kedelapan. Pengisian awal hanya dilakukan sekali; selanjutnya matriks hasil setiap iterasi menjadi masukan iterasi berikutnya.

Setiap iterasi dimulai dengan pertukaran baris berindeks 6 dengan kolom berindeks 6. Untuk setiap $i = 0, \ldots, 15$, elemen $B_{6,i}$ ditukar dengan $B_{i,6}$, bukan disalin satu arah. Setelah seluruh pertukaran tersebut selesai, proses yang sama diterapkan pada baris dan kolom berindeks 7. Elemen diagonal tetap pada posisinya. Pasangan posisi $(6, 7)$ dan $(7, 6)$ ditukar pada kedua tahap sehingga kembali ke susunan sebelum pertukaran. Urutan ini merupakan definisi transposisi parsial yang digunakan dalam implementasi.

Selanjutnya, setiap baris digeser melingkar enam posisi ke kanan dan setiap kolom digeser melingkar tujuh posisi ke bawah. Pergeseran bekerja pada posisi byte, bukan pada bit di dalam byte. Byte yang melewati tepi matriks masuk kembali dari tepi berlawanan. Jika

$T^{(t)}$ adalah matriks setelah kedua tahap pertukaran pada iterasi t, hasil pergeserannya adalah

$$B^{(t+1)}_{(r+7) \bmod 16,\,(c+6) \bmod 16} = T^{(t)}_{r,c}. \tag{3.2}$$

Kode menggunakan buffer terpisah untuk menggabungkan kedua pergeseran tersebut tanpa menimpa elemen yang belum dipindahkan. Seluruh rangkaian pertukaran dan pergeseran diulang 67 kali, yaitu untuk $t = 0, \ldots, 66$.

Alur pembentukan tabel dapat diringkas sebagai berikut:

Matriks awal 16×16 berisi 0–255 per baris
↓
Tukar baris–kolom 6, lalu tukar baris–kolom 7
↓
Geser seluruh baris ke kanan 6 dan seluruh kolom ke bawah 7
↓
Ulangi pertukaran dan pergeseran hingga 67 iterasi
↓
Baca matriks akhir per baris menjadi tabel S_{67}

Matriks akhir dibaca kembali per baris menjadi tabel satu dimensi:

$$S_{67}[x] = B^{(67)}_{\lfloor x/16 \rfloor,\, x \bmod 16}, \qquad x = 0, \ldots, 255. \tag{3.3}$$

Urutan ini berbeda dari pengisian state enkripsi 4×4 yang menggunakan urutan per kolom. Tabel 3.2 menampilkan contoh pasangan substitusi dan invers setelah 67 iterasi; seluruh nilainya ditulis dalam heksadesimal.

Tabel 3.2. Contoh hasil S-box baru dan pemetaan inversnya.

Masukan x	$y = S_{67}[x]$	$S_{67}^{-1}[y]$
00	54	00
01	91	01
02	fc	02
03	39	03

Tabel invers dibentuk oleh `generateInverseSbox67` dengan mengunjungi setiap pasangan x dan $y = S_{67}[x]$, kemudian menyimpan x pada indeks y dalam tabel invers:

$$S_{67}^{-1}[S_{67}[x]] = x. \tag{3.4}$$

Matriks awal memuat setiap nilai byte tepat sekali, sedangkan pertukaran dan pergeseran melingkar hanya memindahkan posisi. Karena itu, tabel akhir tetap memiliki 256 keluaran unik dan invers yang terdefinisi untuk setiap byte.

S-box dan inversnya dibuat sekali saat inisialisasi paket, bersifat tetap, dan tidak

bergantung pada kunci. Enkripsi menggunakan $s'_{r,c} = S_{67}[s_{r,c}]$, sedangkan dekripsi menggunakan tabel invers. S-box yang sama juga dipakai pada substitusi dalam ekspansi kunci.

3.2.2. ShiftRows67

Setiap baris r genap diputar ke kanan sejauh $o_r = (6+r) \bmod 4$. dan baris ganjil sejauh $o_r = (7+r) \bmod 4$

Tabel 3.3. Offset permutasi baris dalam implementasi.

Baris r	0	1	2	3
Offset kanan genap o_r	2	3	0	1
Offset kanan ganjil o_r	1	2	3	0

Inversnya memutar baris ke kiri sejauh offset yang sama.

3.2.3. MixColumns67

Setiap kolom dikalikan dengan matriks M pada GF(2^8). Seluruh elemen berikut ditulis dalam heksadesimal:

$$M = \begin{bmatrix} \texttt{67} & \texttt{06} & \texttt{07} & \texttt{07} \\ \texttt{06} & \texttt{67} & \texttt{06} & \texttt{07} \\ \texttt{07} & \texttt{06} & \texttt{67} & \texttt{06} \\ \texttt{07} & \texttt{07} & \texttt{06} & \texttt{67} \end{bmatrix}, \quad M^{-1} = \begin{bmatrix} \texttt{7a} & \texttt{c5} & \texttt{a0} & \texttt{23} \\ \texttt{c5} & \texttt{fa} & \texttt{c6} & \texttt{a0} \\ \texttt{a0} & \texttt{c6} & \texttt{fa} & \texttt{c5} \\ \texttt{23} & \texttt{a0} & \texttt{c5} & \texttt{7a} \end{bmatrix}. \tag{3.5}$$

Penjumlahan medan menggunakan XOR. Perkalian menggunakan pergeseran bit dan reduksi dengan konstanta `1b` setelah bit tertinggi melimpah. Matriks dipilih karena invertable dan memiliki elemen2 identik dengan 6 dan 7

Matriks dipakai agar setiap keluaran kolom bergantung pada keempat byte masukannya, sementara inversnya memungkinkan dekripsi. Pemeriksaan pada medan yang sama menghasilkan $MM^{-1} = I$ dan seluruh 69 minor persegi bernilai tidak nol. Dengan demikian, M bersifat MDS dengan branch number byte 5: jumlah byte aktif pada masukan dan keluaran suatu perbedaan nonzero minimal lima.

3.2.4. Bit-Plane Transpose (BPT)

Sesudah MixColumns67, state diserialisasi menjadi 16 byte dalam urutan kolom. Bit dalam setiap byte diberi indeks dari MSB ke LSB. Untuk indeks byte $i \in [0, 15]$ dan bit $b \in [0, 7]$, pemetaan adalah

$$p = 8i + b, \qquad p' = 3(16b + i) \bmod 128, \qquad Y[p'] = X[p]. \tag{3.6}$$

Invers menghitung $q = 43p' \bmod 128$, lalu $i = q \bmod 16$, $b = \lfloor q/16 \rfloor$, dan $p = 8i + b$. Karena $3 \cdot 43 \equiv 1 \pmod{128}$, pemetaan ini merupakan permutasi. Transformasi ini bertujuan untuk mengacak urutan bit pada bytes.

3.2.5. Feistel Cross-Mixing (FCM)

Ide untuk transformasi ini adalah memasukkan jaringan Feistel ke dalam Substitution-Permutation Network. FCM hanya diterapkan pada putaran genap. Delapan byte pertama dan terakhir ditafsirkan sebagai L dan R, masing-masing integer 64 bit big-endian. Empat langkah internal menjalankan

$$F_j(x) = \mathrm{ROTL}_{64}(x, a_j) \oplus (x \,\&\, \mathrm{ROTL}_{64}(x, b_j)) \oplus \mathrm{ROTL}_{64}(x, c_j), \tag{3.7}$$

$$(L', R') = (R, L \oplus F_j(R)). \tag{3.8}$$

Operator & menyatakan AND bitwise. Tripel rotasi untuk $j = 1, 2, 3, 4$ berturut-turut adalah $(9, 21, 37)$, $(13, 29, 43)$, $(7, 17, 31)$, dan $(11, 23, 41)$. Invers memproses langkah 4 sampai 1, dengan $(L, R) = (R' \oplus F_j(L'), L')$. Fungsi F_j tidak perlu bijektif agar langkah Feistel dapat dibalik. Konstanta dan kekuatan FCM masih bersifat eksperimental.

3.2.6. Penjumlahan Modulo Byte

Putaran r menggunakan subkunci Q_{2r-1} untuk setiap byte:

$$y_i = (x_i + (Q_{2r-1})_i) \bmod 256, \qquad x_i = (y_i - (Q_{2r-1})_i) \bmod 256. \tag{3.9}$$

Carry hanya merambat di dalam satu byte. Operasi ini juga tidak mengakibatkan pencampuran antarbyte dan tidak memperbesar entropi kunci utama.

3.2.7. AddRoundKey67

Untuk subkunci XOR Q_{2r} pada putaran r, setiap elemen diperbarui sebagai

$$s'_{u,c} = s_{u,c} \oplus (Q_{2r})_{4c+u} \oplus 43. \tag{3.10}$$

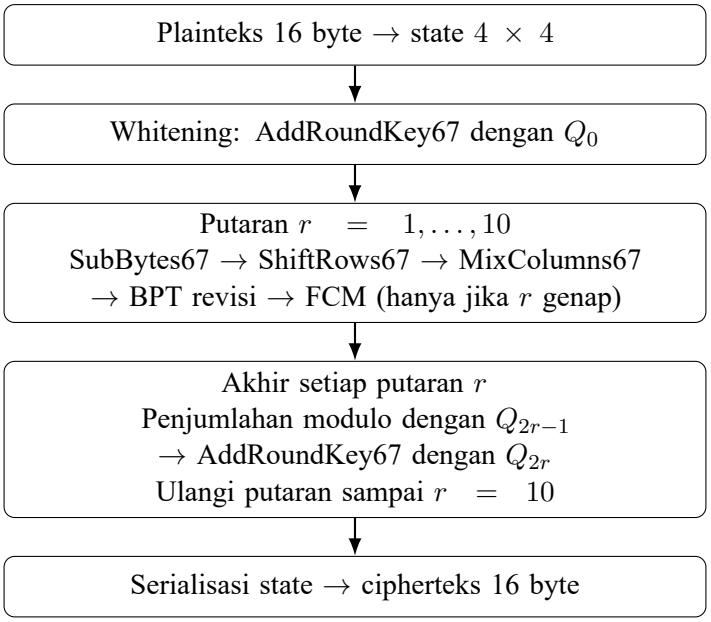
Indeks u adalah baris state. Transformasi ini merupakan invers dirinya sendiri. Whitening awal memakai rumus yang sama dengan Q_0.

3.3. Alur Enkripsi dan Dekripsi

Enkripsi dimulai dengan whitening menggunakan Q_0, kemudian menjalankan sepuluh putaran sesuai Gambar 3.1. Terdapat pola six-seven sepanjang enkripsi. Putaran ganjil memiliki enam operasi dan putaran genap tujuh operasi. Putaran 10 tetap memakai MixColumns67 dan FCM; tidak ada pengecualian putaran terakhir.

Dekripsi memproses $r = 10, \ldots, 1$: AddRoundKey67 dengan Q_{2r}, pengurangan modulo dengan Q_{2r-1}, invers FCM jika r genap, invers BPT, InverseMixColumns67, InverseShiftRows67, dan InverseSubBytes67. Terakhir, whitening dibalik dengan AddRoundKey67 menggunakan Q_0.

Sebagai contoh, blok `Embark at dawn!` diikuti newline (`0a`) dienkripsi dengan kunci `00112233445566778899aabbccddeeff`. Dalam urutan byte per kolom, state setelah



Gambar 3.1. Alur enkripsi satu blok sesuai implementasi saat ini.

whitening adalah `063f0311757d0555bffa8d99f8f08cb6`, sedangkan cipherteks akhirnya `d451087b1554c1abbc5bd05b511c4184`. Hasil tersebut cocok dengan referensi independen, termasuk seluruh state per operasi.

3.4. Pembentukan Kunci Putaran

Kunci utama dibagi menjadi empat word $w_0, \ldots, w_3$, masing-masing empat byte. Ekspansi menghasilkan 84 word. Untuk $i = 4, \ldots, 83$, mula-mula $t = w_{i-1}$. Jika $i \bmod 4 = 0$, t dirotasi satu byte ke kiri, setiap byte disubstitusi dengan S_{67}, lalu byte pertama di-XOR dengan $\text{Rcon}_{i/4-1} \oplus \texttt{67}$. Selanjutnya,

$$w_i = w_{i-4} \oplus t, \qquad Q_j = w_{4j} \| w_{4j+1} \| w_{4j+2} \| w_{4j+3}, \quad 0 \le j \le 20. \tag{3.11}$$

Rcon dimulai dari `01` dan setiap nilai berikutnya merupakan perkalian dengan `02` pada medan yang sama. Dua puluh nilai Rcon adalah `01`, `02`, `04`, `08`, `10`, `20`, `40`, `80`, `1b`, `36`, dilanjutkan `6c`, `d8`, `ab`, `4d`, `9a`, `2f`, `5e`, `bc`, `63`, `c6`. Simbol $\|$ menyatakan konkatenasi byte; Q_0 adalah kunci utama. Putaran r memakai Q_{2r-1} untuk penjumlahan dan Q_{2r} untuk XOR.

Untuk kunci contoh di atas, $w_0 =$ `00112233` dan $w_3 =$ `ccddeeff` menghasilkan $w_4 =$ `1ae7611c`. Word w_4 sampai w_7 membentuk subkunci penjumlahan putaran pertama $Q_1 =$ `1ae7611c5eb2076bd62badd01af6432f`, diikuti subkunci XOR $Q_2 =$ `aeda785cf0687f372643d2e73cb591c8`. Rotasi word mengubah posisi byte, substitusi menambah nonlinieritas, dan Rcon membedakan tahap ekspansi. Konstanta `67` dan `43` memberi identitas rancangan. Penjumlahan menyediakan carry dalam byte, BPT menyusun ulang bit, dan FCM menambah pencampuran nonlinier antarhalf state.

BAB IV
IMPLEMENTASI DAN SIMULASI

4.1. Frontend, Backend, dan CLI

Antarmuka web menggunakan React, TypeScript, dan Vite. Pengguna dapat mengetik teks atau mengunggah berkas, memilih mode serta padding, memasukkan kunci dan IV, lalu mengunduh hasil enkripsi atau dekripsi. Grafik histogram dan entropi dihitung dari byte masukan/keluaran; pengujian avalanche meminta hasil dari backend.

Backend Go menerima JSON untuk teks dan multipart untuk berkas melalui `/api/cipher/process`. Cipher diimplementasikan pada `src/cipher/brainrot/`, sedangkan controller menangani pembacaan data, padding, pemanggilan mode, dan respons hex/base64. Endpoint `/api/cipher/avalanche` menyediakan perubahan bit per putaran dan `/api/cipher/trace` menyediakan state untuk visualisasi. Frontend saat ini memiliki fallback simulasi XOR ketika API gagal; fallback tersebut bukan Brainrot dan tidak digunakan pada pengujian laporan.

Sebagai antarmuka pendukung, CLI `crypt` mengenkripsi argumen teks atau mendekripsi hex, sedangkan `analyze` membaca berkas dan menulis statistik JSON. Keduanya dijalankan dengan `go -C src run ./cmd/...`.

4.2. Demonstrasi Teks dan Berkas Biner

Alur penggunaan web adalah mengisi kunci 16 byte, memilih mode, memasukkan IV 16 byte untuk mode selain ECB, dan memilih PKCS#7. Setelah berkas terenkripsi diunduh, unggah kembali berkas tersebut dan pilih dekripsi menggunakan kunci, mode, padding, dan IV yang sama. Berkas hasil dekripsi dapat diunduh dalam bentuk byte aslinya.

Pengujian demonstrasi mengirim permintaan multipart ke endpoint backend melalui server HTTP lokal sementara yang memakai controller yang sama seperti program web. Jadi, hasil berikut berasal dari Brainrot, bukan fallback browser. Demonstrasi dapat diulang dari root repositori:

```
go -C src run ../scripts/report_demonstration.go
```

Kode 4.1. Menjalankan demonstrasi dan pemeriksaan pemulihan data.

Untuk contoh CBC, teks `Halo Brainrot` dienkripsi dengan parameter berikut, kemudian dipulihkan menjadi teks yang sama.

Kunci (hex)	`00112233445566778899aabbccddeeff`
IV (hex)	`000102030405060708090a0b0c0d0e0f`
Badan cipherteks (hex)	`5bfb0e4535dad1b36014799ee71488eb`

Tabel 4.1. Pemulihan berkas melalui endpoint HTTP dengan PKCS#7.

Mode	Teks (byte)	Biner (byte)	Badan teks / biner (byte)	Byte hasil dekripsi sama
ECB	404	1027	416 / 1040	Ya / Ya
CBC	404	1027	416 / 1040	Ya / Ya
CFB	404	1027	404 / 1027	Ya / Ya
OFB	404	1027	404 / 1027	Ya / Ya
CTR	404	1027	404 / 1027	Ya / Ya

Masukan teks adalah `repeated-text.txt` sepanjang 404 byte dan biner `byte-pattern.bin` sepanjang 1027 byte. Seluruh sepuluh kombinasi berkas/mode menghasilkan respons HTTP 200, kesamaan byte langsung, serta SHA-256 masukan dan hasil dekripsi yang sama.

4.3. Justifikasi dan Bukti Padding

PKCS#7 dipilih karena dapat mengembalikan panjang pesan tanpa menghilangkan byte nol asli. ECB/CBC menambahkan $p = 16 - (L \bmod 16)$ byte, masing-masing bernilai p, termasuk satu blok penuh ketika L sudah kelipatan 16. Dekripsi memeriksa nilai dan seluruh byte padding. CFB, OFB, dan CTR menggunakan sebagian keystream untuk potongan terakhir, sehingga tidak memerlukan padding.

Tabel 4.2. Pemeriksaan batas blok pada CBC dengan PKCS#7.

Panjang masukan (byte)	0	1	15	16	17
Byte padding	16	15	1	16	15
Badan cipherteks (byte)	16	16	16	32	32

Masukan 32 byte juga dipulihkan identik setelah memperoleh padding 16 byte dan badan cipherteks 48 byte. Padding tidak valid ditolak dengan HTTP 400. Opsi ANSI X9.23 dan zero padding tersedia pada web [7], tetapi simulasi memakai PKCS#7. Zero padding tidak dipilih karena data asli `01 00` dipulihkan sebagai `01` oleh implementasi saat ini; byte nol akhir tidak dapat dibedakan dari padding.

4.4. Pengelolaan IV dan Counter

ECB tidak memakai IV. CBC, CFB, dan OFB menggunakan IV 16 byte; CTR menafsirkan 16 byte tersebut sebagai pencacah awal big-endian. IV CBC/CFB harus tidak dapat diprediksi, sedangkan IV OFB dan blok pencacah CTR harus unik untuk kunci yang sama [3]. Karena itu, IV/counter perlu dibuat baru untuk pesan berbeda dan nilai aslinya harus tersedia saat dekripsi. CTR menolak pelimpahan pencacah apabila masih ada blok yang perlu diproses.

CLI memakai `crypto/rand` dan menyertakan IV di depan cipherteks sebagai $\mathrm{IV} \| C$. Web/API saat ini menyimpan IV terpisah melalui isian pengguna; badan berkas hasil belum memuat IV, mode, atau padding. Pengujian memakai IV eksplisit yang sama saat

enkripsi dan dekripsi. Jika IV API dikosongkan, backend membuat nilai acak tetapi tidak mengembalikannya; pengujian OFB pada jalur ini gagal memulihkan pesan. Dengan demikian, demonstrasi memakai IV eksplisit, dan penyimpanan IV otomatis masih perlu diperbaiki.

4.5. Bukti Pemulihan Data dan Batas Integritas

Kesamaan byte dan SHA-256 pada Tabel 4.1 menunjukkan pemulihan tanpa perubahan untuk masukan dan parameter yang diuji. Untuk menilai batas verifikasi saat ini, badan cipherteks teks diubah satu bit pada byte pertama, dipotong satu byte, atau didekripsi dengan kunci/IV yang berbeda.

Tabel 4.3. Pengujian perubahan data.

Kasus	ECB	CBC	CFB	OFB	CTR
Cipherteks diubah 1 bit	200	200	200	200	200
Cipherteks dipotong 1 byte	400	400	200	200	200
Kunci berbeda	400	400	200	200	200
IV berbeda	—	200	200	200	200

Respons 400 pada ECB/CBC berasal dari pemeriksaan panjang atau padding; penolakan pada sampel ini tidak menjamin semua manipulasi akan terdeteksi. Respons 200 pada data yang berubah menunjukkan bahwa program belum memverifikasi autentikasi integritas. Kunci terlalu pendek juga ditolak HTTP 400, tetapi belum ada MAC/tag untuk menolak perubahan isi berkas secara menyeluruh. Bukti yang tersedia adalah pemulihan data pada round-trip, bukan jaminan integritas terhadap pihak yang memodifikasi data.

BAB V

ANALISIS KEAMANAN

5.1. Konfigurasi dan Reproduksibilitas

Pengujian memakai Brainrot, fixture satu blok sepanjang 16 byte, teks `repeated-text.txt` sepanjang 404 byte, dan biner `byte-pattern.bin` sepanjang 1027 byte. Konfigurasi utama adalah 256 percobaan, seed 42, serta kunci `00112233445566778899aabbccddeeff`. Seed hanya digunakan untuk eksperimen; IV pada CLI enkripsi dibangkitkan secara terpisah.

```
go -C src run ./cmd/analyze -cipher brainrot -modes all \
  -trials 256 -seed 42 -output ../reports/brainrot/text-seed42.json
go -C src test ./...
```

Kode 5.1. Menjalankan analisis dari root repositori.

Laporan JSON menyimpan ukuran blok dan kunci, seed, jumlah percobaan, avalanche satu blok, hasil round-trip per mode, entropi, histogram 256 bin, dan avalanche pesan utuh. Pengukuran mode hanya memakai badan cipherteks; IV tidak masuk histogram atau penyebut persentase perubahan bit.

Hasil yang ditampilkan berasal dari `reports/brainrot/text-seed42.json` dan `reports/brainrot/binary-seed42.json`. Fixture biner `src/testdata/fixtures/byte-pattern.bin` terdiri atas nilai byte 0–255 yang diulang empat kali, diikuti `00, ff, 80`; totalnya 1027 byte. Manifest pada folder hasil mencatat kunci, seed, ukuran dan SHA-256 fixture, versi Go, sistem operasi, serta asal pengukuran. Dua eksperimen tambahan memakai seed 1337 dengan kunci nol pada teks dan kunci semua satu pada biner.

5.2. Avalanche Effect Satu Blok

Pada setiap percobaan, satu bit plainteks diubah dengan kunci tetap; secara terpisah, satu bit kunci diubah dengan plainteks tetap. Percobaan pertama memakai fixture satu blok, sedangkan berikutnya memakai blok pseudorandom dari seed yang tercatat. Posisi bit digilir melalui seluruh 128 bit masukan; setidaknya 128 percobaan diperlukan untuk mencakup setiap posisi. Keluaran mencatat rata-rata, minimum, maksimum, serta rata-rata dan jumlah percobaan per posisi bit.

Tabel 5.1. Hasil avalanche Brainrot satu blok, 256 percobaan dan seed 42.

Perubahan	Percobaan	Rata-rata (%)	Minimum (bit)	Maksimum (bit)
1 bit plainteks	256	49.924	49	78
1 bit kunci	256	49.817	47	81

Rata-rata perubahan adalah 49,924% untuk bit plainteks dan 49,817% untuk bit kunci,

mendekati separuh keluaran. Setiap posisi bit hanya diuji dua kali; rata-rata per posisi berkisar 55–73 bit untuk plainteks dan 53,5–73 bit untuk kunci. Variasi ini belum cukup untuk menyimpulkan bahwa setiap bit keluaran berubah dengan peluang yang sama. Dengan seed 1337, kunci nol menghasilkan rata-rata 49,780%/49,597%, sedangkan kunci semua satu menghasilkan 50,208%/49,905% (plainteks/kunci).

5.3. Avalanche Effect pada Seluruh Mode

Untuk pesan utuh, posisi bit plainteks dipilih pseudorandom dan posisi bit kunci digilir. Pasangan enkripsi sebelum dan sesudah perubahan menggunakan IV yang sama agar perbedaan berasal dari bit yang diubah. Penyebutnya adalah jumlah bit seluruh badan cipherteks, termasuk padding ECB/CBC, sehingga berbeda dari pengujian satu blok.

Tabel 5.2. Avalanche pesan teks 404 byte, 256 percobaan dan seed 42.

Mode	Panjang badan (byte)	Perubahan plainteks (%)	Perubahan kunci (%)
ECB	416	1.922	49.967
CBC	416	25.957	50.127
CFB	404	24.131	50.018
OFB	404	0.031	50.079
CTR	404	0.031	50.055

Pada OFB dan CTR dengan kunci serta IV tetap, satu bit plainteks yang diubah menghasilkan satu bit cipherteks yang berubah karena operasi XOR dengan keystream yang sama. Pada ECB perubahan terbatas pada blok terkait; pada CBC dan CFB pengaruhnya bergantung pada posisi perubahan dan umpan balik. Interpretasi hasil harus membedakan sifat mode dari difusi cipher satu blok.

Pada teks, perubahan plainteks ECB hanya 1,922% karena pengaruhnya terbatas pada satu blok dari badan 416 byte. CBC dan CFB mencapai 25,957% dan 24,131% karena perubahan dapat merambat ke blok berikutnya; posisi bit yang dipilih menentukan banyaknya blok yang terkena pengaruh. OFB dan CTR sama-sama 0,031%, yaitu satu bit dari 3232 bit badan. Pada biner yang lebih panjang, persentase ECB turun menjadi 0,772% dan OFB/CTR menjadi 0,012%; CBC/CFB tetap sekitar 25%. Sebaliknya, perubahan kunci menghasilkan sekitar 50% pada kelima mode karena kunci memengaruhi semua pemanggilan cipher. Persentase perubahan plainteks yang kecil pada OFB/CTR merupakan sifat mode, bukan kegagalan avalanche cipher satu blok.

5.4. Analisis Entropi

Entropi teks meningkat dari 4,860 menjadi 7,232 bit/byte pada ECB dan 7,466–7,542 pada mode lain. Pada biner, plainteks sudah mendekati 8 bit/byte, tetapi ECB hanya mencapai 7,128 karena blok berulang tetap menghasilkan cipherteks berulang. Mode lain mencapai 7,789–7,812. Entropi tidak harus meningkat: nilainya bergantung pada distribusi awal, ukuran sampel, pengulangan blok, dan tambahan padding ECB/CBC.

Tabel 5.3. Entropi empiris untuk fixture teks, seed 42; ukuran adalah badan cipherteks.

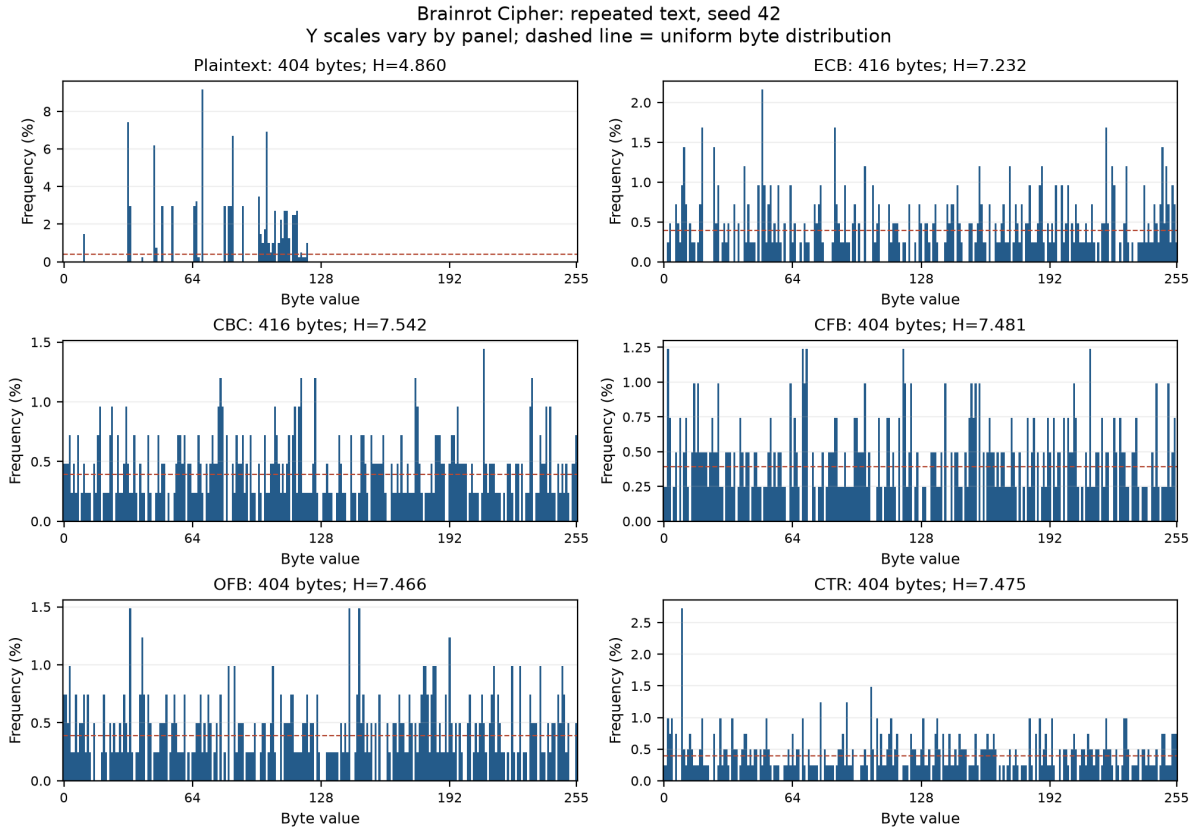
Mode	Ukuran sampel (byte)	Plainteks (bit/byte)	Cipherteks (bit/byte)
ECB	416	4.860	7.232
CBC	416	4.860	7.542
CFB	404	4.860	7.481
OFB	404	4.860	7.466
CTR	404	4.860	7.475

Tabel 5.4. Entropi empiris untuk fixture biner, seed 42.

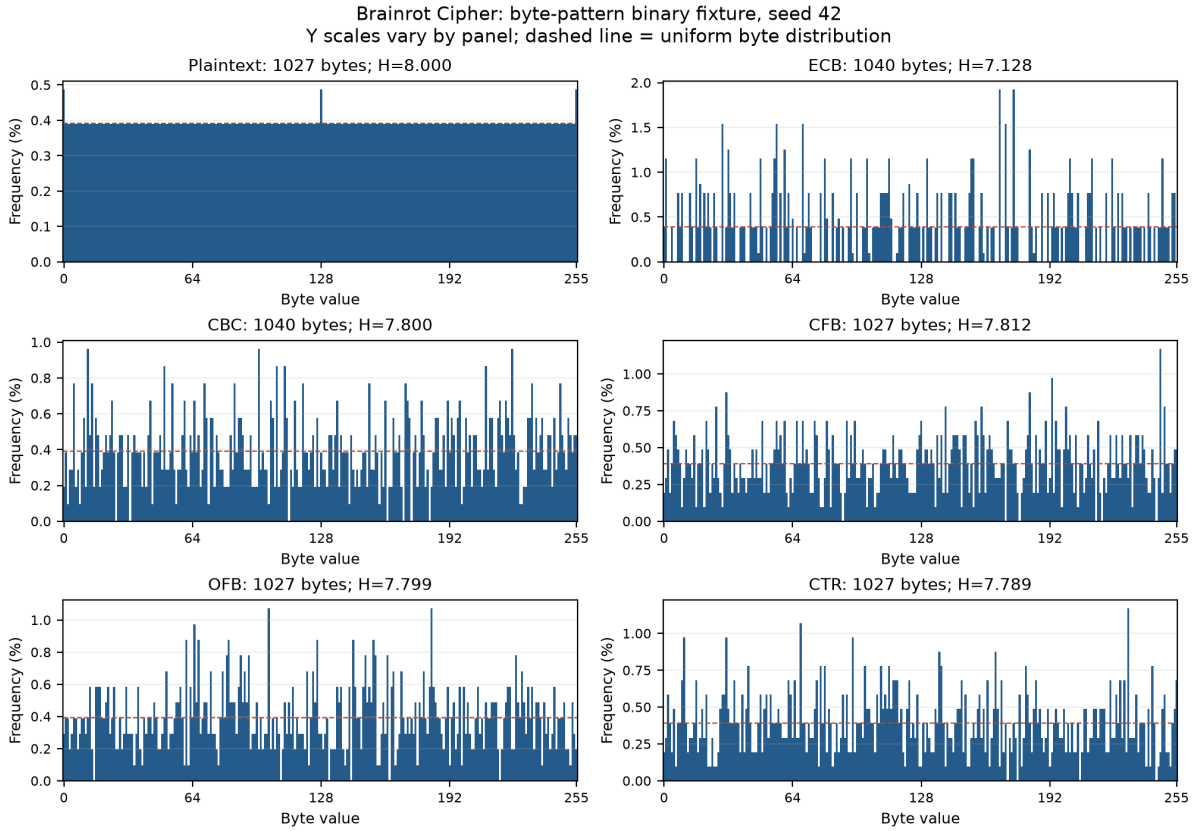
Mode	Ukuran badan (byte)	Plainteks (bit/byte)	Cipherteks (bit/byte)
ECB	1040	8.000	7.128
CBC	1040	8.000	7.800
CFB	1027	8.000	7.812
OFB	1027	8.000	7.799
CTR	1027	8.000	7.789

5.5. Analisis Histogram

Grafik memakai proporsi byte agar panjang badan yang berbeda dapat dibandingkan. Skala y berbeda antar panel dan tercantum pada setiap sumbu. Pada teks, ECB memiliki 75 nilai byte yang tidak muncul dan puncak frekuensi 9 kemunculan, sedangkan mode lain memiliki 43–54 nilai yang tidak muncul. Perbedaannya lebih jelas pada biner: ECB memiliki 94 nilai yang tidak muncul dan puncak 20 kemunculan; mode lain hanya 4–7 nilai yang tidak muncul. Hal ini selaras dengan blok masukan yang berulang dan pemetaan ECB yang independen per blok. Sebaran CBC, CFB, OFB, dan CTR lebih mendekati seragam pada sampel biner ini, tetapi batang tidak harus sama tinggi karena jumlah byte terbatas. Histogram juga tidak menampilkan urutan byte, sehingga kemerataan saja tidak membuktikan ketiadaan pola.



Gambar 5.1. Histogram fixture teks dan cipherteks lima mode pada seed 42; sumbu y memakai skala berbeda dan garis putus-putus menandai distribusi seragam.



Gambar 5.2. Histogram fixture biner dan cipherteks lima mode, seed 42; sumbu y memakai skala berbeda.

5.6. Pengujian Tambahan dan Keterbatasan

5.6.1. Kebenaran Implementasi

Pengujian mode memakai `TEST-ONLY-XOR`, yaitu cipher sederhana khusus pengujian yang hasilnya mudah dihitung, untuk memeriksa urutan chaining antarblok. Pengujian dengan Brainrot kemudian memeriksa pemulihan data menggunakan algoritma sebenarnya. Empat pasangan kunci dan plainteks dicocokkan dengan implementasi referensi Python yang terpisah: cipherteks akhir, 21 subkunci, dan 66 state perantara per kasus harus sama. Pencocokan state membantu menemukan tahap yang keliru, meskipun enkripsi dan dekripsi masih saling membalik.

Pengujian juga memeriksa invers setiap operasi, masukan berpola dan pseudorandom, serta buffer masukan/keluaran yang saling tumpang tindih. Seluruh 128 posisi bit BPT dan 65.536 pasangan nilai byte/subkunci penjumlahan diperiksa. Kelima mode diuji pada masukan kosong dan batas blok, beserta penolakan padding tidak valid pada mode yang memakainya dan penolakan pelimpahan pencacah CTR. Hasil ini mendukung kesesuaian implementasi dengan rancangan pada kasus yang diuji.

Pemeriksaan rank membuktikan branch number byte diferensial minimal 5 (36.416 pasangan posisi), sedangkan untuk masker linier nilainya tepat 4 (4.096 pasangan posisi dan contoh mask pada `TestLinearMaskBranchWitness`). Sifat MDS pada MixColumns saja tidak menjamin batas yang sama setelah BPT. Pemeriksaan ini mencakup lapisan linier, bukan keamanan cipher sepuluh putaran atau autentikasi data.

BAB VI

KESIMPULAN DAN SARAN PENGEMBANGAN

6.1. Kesimpulan

Berdasarkan perancangan, implementasi, serta analisis yang telah dilakukan terhadap Brainrot Cipher, ditarik beberapa kesimpulan sebagai berikut:

1. **Pemenuhan Spesifikasi Perancangan Cipher:** Algoritma Brainrot berhasil dirancang dan diimplementasikan sebagai *block cipher* simetris 128 bit dengan panjang kunci 128 bit, 10 putaran, serta 21 subkunci terikat melalui prosedur *key schedule*. Struktur berbasis *Substitution–Permutation Network* (SPN) memadukan substitusi nonlinier Kotak-S S_{67} (permutasi matriks 16×16 sepanjang 67 iterasi), pergeseran baris ShiftRows67, difusi kolom linear MixColumns67 menggunakan matriks MDS pada GF(2^8) dengan *branch number* 5, Bit-Plane Transpose (BPT), langkah nonlinier Feistel Cross-Mixing (FCM) pada putaran genap, serta kombinasi subkunci penjumlahan modulo byte dan XOR AddRoundKey67. Seluruh komponen transformasi terbukti dapat dibalik (*invertible*) dan kesesuaian alurnya telah divalidasi menggunakan vektor uji independen serta pencocokan 66 *state* perantara.
2. **Implementasi Mode Operasi dan Penanganan Padding:** Kelima mode operasi (ECB, CBC, CFB, OFB, dan CTR) berhasil diimplementasikan untuk memproses data teks (404 byte) dan berkas biner (1027 byte). Skema padding PKCS#7 pada mode ECB dan CBC berhasil menangani sisa byte maupun pesan yang sudah tepat kelipatan blok, serta menolak padding tidak valid dengan respons HTTP 400. Mode CFB, OFB, dan CTR memproses sisa byte melalui operasi XOR dengan potongan *keystream* tanpa memerlukan padding. Seluruh sepuluh kombinasi pengujian *round-trip* menunjukkan pemulihan data yang identik dengan masukan aslinya melalui kesamaan byte langsung dan nilai *digest* SHA-256.
3. **Batas Verifikasi dan Autentikasi Integritas:** Keberhasilan pemulihan data pada pengujian *round-trip* membuktikan invertibilitas algoritma dan kesesuaian implementasi, tetapi tidak menjamin autentikasi integritas cipherteks. Percobaan manipulasi satu bit pada cipherteks menghasilkan respons dekripsi berstatus HTTP 200 tanpa deteksi kegagalan integritas, membuktikan bahwa penolakan galat saat ini hanya bertumpu pada pemeriksaan batas blok dan struktur padding. Implementasi belum menyertakan mekanisme autentikasi seperti *Message Authentication Code* (MAC) atau skema *Authenticated Encryption*.
4. **Karakteristik Statistik dan Keterbatasannya:**
    - *Avalanche effect* satu blok menghasilkan rata-rata perubahan bit sebesar 49,976% untuk pembalikan satu bit plainteks dan 49,557% untuk pembalikan satu bit kunci

(256 percobaan, seed 42), yang mendekati nilai ideal 50% pada kriteria difusi empiris.

- Pada pesan utuh, perambatan perubahan bit plainteks mencerminkan sifat masing-masing mode operasi: ECB hanya mencapai 1,933% karena pengaruh perubahan terbatas pada satu blok; CBC dan CFB mencapai 26,001% dan 24,164% akibat mekanisme *chaining* dan umpan balik; sedangkan OFB dan CTR menghasilkan tepat satu bit perubahan (0,031%) karena sifat *keystream* independen. Sebaliknya, pembalikan bit kunci menghasilkan perubahan sekitar 50% pada kelima mode (49,908%–50,054%) karena kunci memengaruhi seluruh pemanggilan cipher.
- Entropi empiris teks meningkat dari 4,860 bit/byte menjadi 7,188 bit/byte pada ECB dan 7,433–7,571 bit/byte pada mode lainnya. Pada berkas biner, mode non-ECB mencapai entropi 7,804–7,817 bit/byte dengan sebaran histogram yang mendekati seragam. Sebaliknya, ECB pada biner hanya mencapai 7,220 bit/byte dengan 86 nilai byte yang tidak muncul akibat blok masukan yang berulang.
- Kemerataan histogram dan tingginya entropi empiris merupakan indikator sebaran frekuensi byte pada sampel uji, tetapi bukan merupakan jaminan ketiadaan pola atau bukti ketahanan terhadap kriptanalisis formal.

6.2. Saran Pengembangan

Berdasarkan hasil evaluasi, keterbatasan pengujian, dan temuan implementasi saat ini, beberapa saran pengembangan yang diajukan adalah sebagai berikut:

1. **Kajian Kriptanalisis Formal:** Melakukan analisis matematis mendalam terhadap ketahanan Brainrot Cipher terhadap kriptanalisis diferensial dan linier. Kajian lanjutan perlu menghitung probabilitas karakteristik diferensial maksimum (*maximum differential characteristic probability*), menyusun tabel profil perbedaan (DDT) dan aproksimasi linier (LAT) Kotak-S S_{67}, serta membuktikan batas bawah jumlah putaran aktif (*active S-boxes*) di bawah pengaruh kombinasi ShiftRows67, MixColumns67, Bit-Plane Transpose (BPT), dan langkah Feistel FCM.
2. **Integrasi Autentikasi Integritas dan Format Berkas:**
   - Mengembangkan mode operasi terotentikasi, seperti GCM (*Galois/Counter Mode*) atau skema *Encrypt-then-MAC* menggunakan HMAC atau KMAC, agar sistem dapat memverifikasi integritas dan keaslian cipherteks serta menolak manipulasi data secara kriptografis.
   - Merancang format wadah berkas terstandardisasi yang menyertakan *header* metadata (pengenal format, identitas mode, skema padding, serta nilai IV acak) sehingga proses dekripsi berkas tidak lagi memerlukan masukan parameter manual pengguna.
3. **Penyempurnaan Antarmuka dan Manajemen Kunci/IV:**
   - Memperbaiki penanganan IV otomatis pada antarmuka web dan API agar nilai

IV acak yang dibangkitkan oleh backend selalu dikembalikan kepada klien atau disematkan di awal berkas keluaran secara konsisten.

- Menghapus mekanisme *fallback* simulasi XOR pada antarmuka peramban dan menggantinya dengan penanganan galat jaringan yang eksplisit guna mencegah kebingungan pengguna terhadap hasil enkripsi.

4. **Optimasi Kinerja dan Evaluasi Saluran Samping:** Menerapkan tabel pra-komputasi (*T-tables*) yang menggabungkan operasi SubBytes67, ShiftRows67, dan MixColumns67 untuk meningkatkan *throughput* pemrosesan perangkat lunak, serta mengevaluasi implementasi terhadap kerentanan saluran samping (*side-channel attacks*), terutama serangan waktu (*timing attack*) pada operasi perkalian medan berhingga $\mathrm{GF}(2^8)$ dan operasi percabangan bit.

DAFTAR PUSTAKA

[1] A. E. R. Xavier dan N. G. Subrata, “Tugas 2 IF4020-Kriptografi Semester 1 Tahun 2026/2027: Membuat Block Cipher Baru,” spesifikasi tugas, 2026. Salinan lokal: `Tugas2-IF4020-2026.md`.

[2] National Institute of Standards and Technology, “Advanced Encryption Standard (AES),” FIPS 197-upd1, 9 Mei 2023. [Daring]. Tersedia: https://doi.org/10.6028/NIST.FIPS.197-upd1.

[3] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Methods and Techniques,” NIST SP 800-38A, Desember 2001. [Daring]. Tersedia: https://doi.org/10.6028/NIST.SP.800-38A.

[4] R. Housley, “Cryptographic Message Syntax (CMS),” RFC 5652, bagian 6.3 dan 9, September 2009. [Daring]. Tersedia: https://www.rfc-editor.org/rfc/rfc5652.

[5] C. E. Shannon, “Communication Theory of Secrecy Systems,” *Bell System Technical Journal*, vol. 28, no. 4, hlm. 656–715, Oktober 1949. [Daring]. Tersedia: https://doi.org/10.1002/j.1538-7305.1949.tb00928.x.

[6] H. M. Heys, “A Tutorial on Linear and Differential Cryptanalysis,” laporan teknis CORR 2001-17, Centre for Applied Cryptographic Research, University of Waterloo, Maret 2001. [Daring]. Tersedia: https://www.engr.mun.ca/~howard/PAPERS/ldc_tutorial.pdf.

[7] D. Ireland, “Padding schemes for block ciphers,” *CryptoSys PKI Pro Manual*, DI Management Services, 2026. [Daring]. Tersedia: https://www.cryptosys.net/pki/manpki/pki_paddingschemes.html.

LAMPIRAN

A. Pranala Repositori

https://github.com/mraifalkautsar/Tugas2-IF4020-2026-K11

[ISI: Cantumkan tag release atau commit yang sesuai dengan hasil laporan.]

B. Video Demo dan Dokumentasi API (Opsional)

Video demo: https://youtu.be/k17aw6r5oJw.

Dokumentasi API menggunakan Swagger UI pada http://localhost:8000/docs/, dengan spesifikasi OpenAPI pada http://localhost:8000/openapi.json. Parameter, keluaran, kesalahan, dan contoh permintaan teks/berkas tersedia pada `docs/api.md` dan README repositori. Dokumentasi mencakup health check, pemrosesan cipher, trace, dan avalanche. *[ISI: Tambahkan pranala Swagger UI publik setelah dokumentasi dipublikasikan.]*

C. Pembagian Tugas Anggota

Tabel 6.1. Isian pembagian tugas kelompok.

Nama	NIM	Kontribusi
Muhammad Ra'if Alkaustar	13523011	*[ISI: Perancangan arsitektur inti cipher, ekspansi subkunci (key schedule), pengujian performa benchmark, penulisan laporan analisis keamanan LaTeX.]*
Muhammad Aditaya Rahmadeni	13523028	*[ISI: Pengembangan antarmuka Web Neo-Brutalist (React + TypeScript + Tailwind), visualisasi metrik analitik (Avalanche, Entropi, Histogram), arsitektur REST API & routing HTTP, kontainerisasi Docker & CI/CD deployment]*
Fityatul Haq Rosyidi	13523116	*[ISI: Implementasi transformasi cipher (generasi tabel S-Box dinamis & ShiftRows), utilitas pengujian Command Line Interface (CLI Crypt), integrasi layanan enkripsi/dekripsi, penyusunan dokumentasi dasar teori.]*

D. Petunjuk Menjalankan

Jalankan dari root repositori dengan Go 1.23 atau lebih baru.

```
go version
go -C src test ./...
go -C src run ./cmd/crypt -op encrypt -mode ECB \
  -key-hex 00112233445566778899aabbccddeeff -input "Halo Brainrot"
go -C src run ./cmd/analyze -cipher brainrot -modes all \
  -trials 256 -seed 42 -output ../reports/brainrot.json
```

Kode 6.1. Perintah verifikasi dan analisis.

Di PowerShell, satukan setiap perintah menjadi satu baris atau gunakan backtick sebagai penyambung baris. README repositori memuat contoh enkripsi/dekripsi dengan variabel PowerShell.

Backend web dijalankan dengan `go -C src run ./cmd/server`, sedangkan frontend memakai `npm --prefix frontend run dev` setelah dependensinya dipasang. Demonstrasi pemulihan data dan kasus perubahan dapat diulang dengan `go -C src run ../scripts/report_demonstration.go`. Pengelolaan IV dan batas verifikasi integritas dijelaskan pada Bab IV.